

Storage Management

StorageClasses, PV/PVC, dynamic vs static provisioning, local-path, real NFS, a real CSI driver, and StatefulSet storage - on one k3s node.

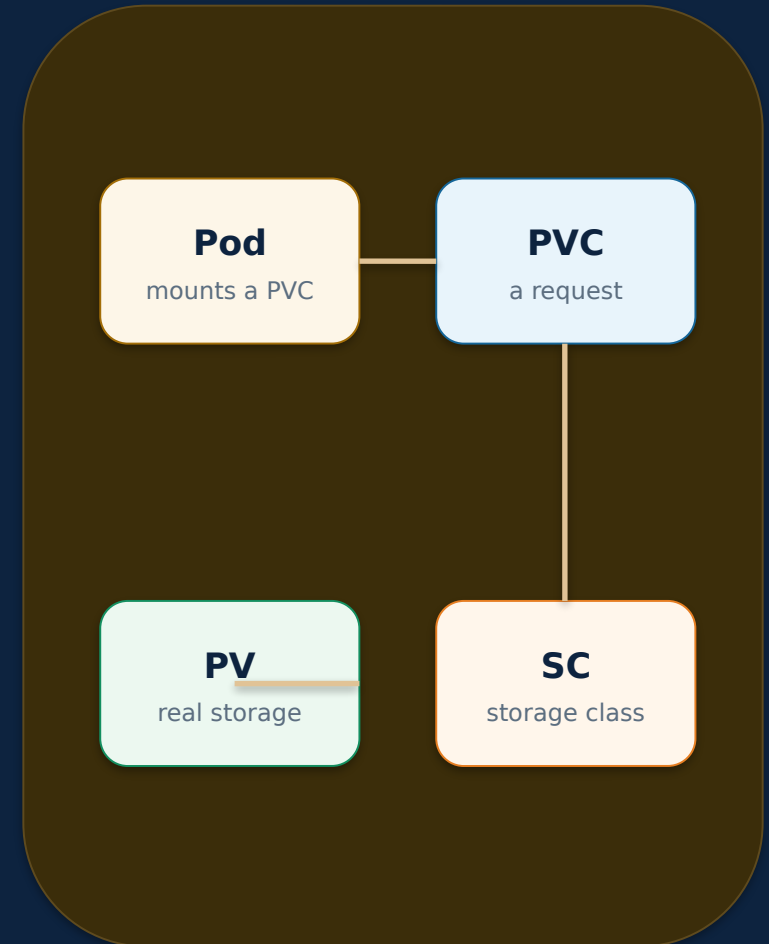
STORAGECLASS

PV / PVC

NFS

CSI DRIVER

STATEFULSET



Where this sits in the course

Day 01

Workloads,
config,
Secrets, RBAC.

Day 02

Least-privilege
ServiceAccounts.

Day 03

Services, DNS,
NetworkPolicy.

Day 04

iptables, Flannel,
CoreDNS
internals.

Day 05

Ingress, TLS,
load balancing,
egress.

Today

What happens to a Pod's data when the Pod doesn't survive - which, eventually, all Pods don't.

```
kubectl get storageclass
```

```
kubectl get pv,pvc -A
```

Single-node scope: what's genuinely real here

Storage needed LESS faking than Ingress did - nothing here is a content-only slide.

CSI Drivers

Can't demo a cloud CSI driver (EBS, PD) on one node - but a REAL, spec-compliant CSI driver (csi-driver-nfs) installs and runs on it. ex04 uses it for real dynamic provisioning.

NFS Storage

No separate storage server exists here, so nfs-kernel-server was installed on the SAME node the cluster runs on. Client and server are the same machine - genuinely working NFS.

Every StorageClass, PV, PVC, and the CSI driver below are real objects on this cluster.

What's verified about this cluster

```
$ kubectl get storageclass
NAME                                PROVISIONER                RECLAIMPOLICY  BINDINGMODE
local-path (default)  rancher.io/local-path      Delete         WaitForFirstConsumer

$ kubectl -n kube-system get pod -l app=local-path-provisioner
local-path-provisioner-77b9867795-ks6kv  1/1  Running

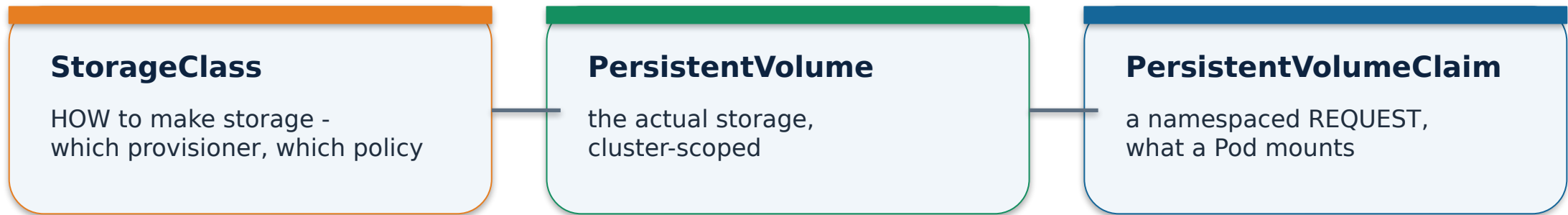
$ systemctl is-active nfs-kernel-server
active

$ kubectl get csidriver
NAME                ATTACHREQUIRED  MODES
nfs.csi.k8s.io      false           Persistent
```

local-path ships with k3s and is the default StorageClass on install. NFS and the CSI driver were added for this module.

Why storage needs its own object model

A Pod's own filesystem dies with the Pod. Kubernetes solves persistence with three cooperating objects, not one.



Key point

A Pod never talks to a PV directly - it mounts a PVC, and the PVC is bound to a PV. That indirection is what lets the same Pod spec work whether the volume came from local-path, NFS, or a cloud disk.

Static vs. dynamic provisioning

	Static	Dynamic
Who creates the PV	an admin, by hand, ahead of time	a provisioner, on demand, when a PVC asks
Matching	PVC's storageClassName + size/mode fits an existing PV	PVC references a StorageClass; provisioner creates a matching PV
This module's example	ex02 (hostPath), ex03 (NFS)	ex01 (local-path), ex04 (CSI NFS)

Both are demonstrated on the SAME underlying storage (NFS: ex03 vs ex04) so the contrast is visible, not just asserted.

local-path-provisioner: dynamic, no extra install

k3s's bundled provisioner (rancher.io/local-path) watches PVCs and creates a directory + PV on demand.

volumeBindingMode: WaitForFirstConsumer delays provisioning until a Pod using the PVC is scheduled.

```
$ kubectl apply -f ex01-local-path-dynamic/pvc.yaml
$ kubectl get pvc data-claim
STATUS: Pending           # normal - no consumer yet

$ kubectl apply -f ex01-local-path-dynamic/pod.yaml
$ kubectl get pvc data-claim
STATUS: Bound             # now that "writer" is scheduled

$ sudo ls /var/lib/rancher/k3s/storage/
pvc-2f03cfda-...-d395f2ab5516_day06-storage_data-claim
```

Deleting and recreating the Pod: the log file kept its old lines and gained new ones - the PVC outlives the Pod.

PV lifecycle & reclaim policy

Available -> Bound -> Released -> (Deleted | Available again)

Reclaim policy	On PVC delete
Delete (local-path's default)	PV AND its data are destroyed automatically
Retain	PV becomes Released; data kept; NOT auto-reusable

```
$ kubectl delete pvc manual-claim      # bound to a Retain PV
$ kubectl get pv manual-pv
STATUS: Released                      # not gone, not Available

$ cat /mnt/day06-manual-pv/marker.txt
written-by-manual-writer              # data intact

$ kubectl patch pv manual-pv --type=json -p ' [{"op": "remove", "path": "/spec/claimRef"} ]'
# only NOW does a new PVC see it as Available
```

NFS: genuine shared storage, on this same node

ex03-nfs-storage/setup-nfs-server.sh does exactly this, in one command:

```
$ sudo apt-get install -y nfs-kernel-server
$ echo '/srv/nfs/k3s-training 10.42.0.0/16(rw,...) \
  192.168.230.0/23(rw,...) 127.0.0.1(rw,...)' | sudo tee /etc/exports
$ sudo exportfs -ra && sudo systemctl enable --now nfs-kernel-server
```

A real quirk found while wiring this up

The export first allowed only the Pod CIDR (10.42.0.0/16) and 127.0.0.1 - mounting failed with "mount.nfs: access denied by server". The in-tree Kubernetes NFS volume plugin has the KUBELET ON THE NODE perform the mount, then binds it into the Pod - so the NFS client connection comes from the node's own IP (192.168.230.103), not a Pod IP. The export had to allow the node's subnet too.

Access modes: what actually differs by backend

Mode	Meaning	local-path/hostPath	NFS
RWO	one node, read-write	yes	yes
ROX	many nodes, read-only	no	yes
RWX	many nodes, read-write	no	yes
RWOP	exactly one Pod (stricter)	yes	yes

```
$ kubectl exec nfs-reader -- cat /data/shared.txt      # sees nfs-writer's file
from-nfs-writer
$ kubectl exec nfs-reader -- sh -c 'echo from-reader >> /data/shared.txt'
$ kubectl exec nfs-writer -- cat /data/shared.txt      # sees it too
from-nfs-writer
from-reader
```

Two INDEPENDENT Pods, one volume, both directions - genuine RWX, verified with real Pods.

CSI drivers: the interface local-path predates

CSI is a gRPC contract between the kubelet and a storage driver - CreateVolume, NodeStageVolume, NodePublishVolume.

Any conformant driver works with any CSI-aware cluster, no code changes.

Ships as: a controller Pod (create/delete volumes) + a node DaemonSet (mount into Pods) + a CSIDriver object.

local-path-provisioner is NOT CSI - it's the older "external provisioner" pattern that predates the CSI spec.

```
$ kubectl get csidriver # before ex04: empty
$ bash ex04-csi-nfs-dynamic/install-csi-driver.sh
$ kubectl get csidriver
NAME              ATTACHREQUIRED  MODES
nfs.csi.k8s.io    false           Persistent
```

CSI in action: dynamic NFS, isolated per claim

```
$ kubectl apply -f ex04-csi-nfs-dynamic/storageclass.yaml # provisioner: nfs.csi.k8s.io
$ kubectl apply -f ex04-csi-nfs-dynamic/pvcs-and-pods.yaml # two PVCs, same class

$ ls /srv/nfs/k3s-training/
pvc-3728c26e-...  pvc-62fd3e43-...  # one dir PER PVC, auto-created

$ kubectl exec csi-writer-a -- ls /data/
                        marker.txt                # can't see claim-b's files
```

```
$ kubectl delete pvc csi-claim-a
$ ls /srv/nfs/k3s-training/
                        pvc-3728c26e-...          # claim-a's directory: gone too
```

No Released PV, no manual claimRef cleanup - the CSI driver did the reclaim itself. Contrast with Slide 8.

StatefulSets: storage with a stable identity

A Deployment's replicas are interchangeable; a StatefulSet's are not.

Each gets a stable name (counter-0, counter-1, ...) and its OWN PVC via volumeClaimTemplates.

Pods are created and deleted in order (0, then 1, then 2, ...) - visible in the rollout, not just asserted.

```
volumeClaimTemplates:
- metadata: { name: data }
  spec:
    accessModes: ["ReadWriteOnce"]
    resources: { requests: { storage: 50Mi } }
```

One PVC per replica

data-counter-0
data-counter-1

created and attached
automatically, no
extra YAML per replica.

Proof: the PVC survives, the Pod doesn't have to

```
$ kubectl exec counter-0 -- cat /data/identity.log
I am counter-0, first boot at ...16:05:13...

$ kubectl delete pod counter-0 --now          # StatefulSet recreates counter-0

$ kubectl exec counter-0 -- cat /data/identity.log
I am counter-0, first boot at ...16:05:13...  # <- original marker, still there
...
      I am counter-0, first boot at ...16:05:34...  # <- second marker, from the restart
```

Two "first boot" lines in ONE file - the container restarted clean, the volume did not.

Storage best-practice checklist

Pick accessModes the StorageClass's backend can actually satisfy - check before writing the PVC.

Decide reclaimPolicy and allowVolumeExpansion AT StorageClass creation - both are hard to retrofit.

Retain is safer but needs an ops process to reclaim; Delete is convenient but irreversible.

kubectl describe pvc - not kubectl get - is where the real provisioning error always shows up.

Give StatefulSets (databases, queues) volumeClaimTemplates, never a single shared PVC across replicas.

Size requests are a REQUEST, not an enforced quota on local-path/NFS - don't rely on the number alone.

Failure drills: real rejections, real messages

```
$ kubectl apply -f ex06-storage-best-practices/drill-rwx-on-localpath.yaml
$ kubectl describe pvc rwx-on-localpath
Warning ProvisioningFailed ... NodePath only supports ReadWriteOnce and
  ReadWriteOncePod (1.22+) access modes

$ kubectl patch pvc data-counter-0 -p '{"spec":{"resources":{"requests":{"storage":"200Mi"}}}}'
Error from server (Forbidden): ... only dynamically provisioned pvc can be
  resized and the storageclass that provisions the pvc must support resize
```

Both come back immediately from the API/controller itself - no timeout to wait out, if you know to describe/read the error.

Troubleshooting sequence

```
kubectl get pvc,pv -n <ns>
kubectl describe pvc <name> -n <ns>           # the real reason is always here
kubectl -n kube-system logs deploy/local-path-provisioner --tail=50
kubectl -n kube-system logs deploy/csi-nfs-controller -c nfs --tail=50
          kubectl get storageclass <name> -o yaml          # reclaimPolicy, expansion, bindingMode
```

Symptom	Likely cause
PVC Pending, no events yet	WaitForFirstConsumer - normal until a Pod uses it
PVC Pending + ProvisioningFailed	access mode the backend can't do, or provisioner not running
PV Released, new PVC also Pending	Retain policy - clear the stale claimRef
Resize Forbidden	no allowVolumeExpansion, or PV wasn't dynamically provisioned
NFS access denied by server	export must allow the node's subnet, not just the Pod CIDR
Two Pods can't share a volume	check accessModes - use NFS/CSI for ReadWriteMany

Hands-on labs — built and verified

Lab	Proves
ex01 local-path dynamic	WaitForFirstConsumer, data survives a Pod restart
ex02 static PV + hostPath	hand-made PV, Retain reclaim, manual claimRef fix
ex03 NFS storage	real NFS server, genuine ReadWriteMany across two Pods
ex04 CSI NFS dynamic	real CSI driver, per-claim isolation, automatic Delete cleanup
ex05 StatefulSet storage	one PVC per replica, reattached (not recreated) after a restart
ex06 best-practice drills	RWX on local-path and a non-expandable resize, both rejected

Same pattern as Days 01-05: manifests + LAB-MANUAL.md (steps + troubleshooting) + OUTPUT.md (real captured output).

RECAP

The Pod dies. The data doesn't have to.

Pod <-> PVC (a request) <-> PV (the actual storage) <-> StorageClass (how to make more).

Static provisioning = you make the PV; dynamic = a provisioner makes it when a PVC asks.

local-path is dynamic but not CSI; csi-driver-nfs is a real CSI driver - both work on one node.

Retain keeps data and needs a human; Delete is automatic and irreversible - decide per StorageClass.

StatefulSets give each replica its own PVC automatically; the Pod can restart without losing state.